Experiment 7: SELECT with GROUP BY, HAVING and ORDER BY

AIM:

To view the records from the tables using SELECT commands with Group By, Having, Order By

Algorithm / Procedure

1. Start the program.
2. Open the MySQL Command Line Client.
3. Select the **Supermarket** database using the `USE` command.
4. Ensure the **Vegetable** table contains records.
5. Use the **GROUP BY** clause to group records based on a column.
6. Use the **HAVING** clause to filter grouped records.
7. Use the **ORDER BY** clause to sort records in ascending or descending order.
8. Display the results.
9. Stop the program.

SQL Queries

GROUP BY

1. **Display the total quantity in each category.**

```
SELECT Category, SUM(Quantity) AS Total_Quantity
FROM Vegetable
GROUP BY Category;
```

```
mysql> SELECT Category, SUM(Quantity) AS Total_Quantity
    -> FROM Vegetable
    -> GROUP BY Category;
+-----------+----------------+
| Category  | Total_Quantity |
+-----------+----------------+
| Vegetable |            540 |
| Root      |             80 |
+-----------+----------------+
2 rows in set (0.03 sec)
```

2. **Display the average price of each category.**

```
SELECT Category, AVG(Price) AS Average_Price
FROM Vegetable
GROUP BY Category;
```

```
mysql> SELECT Category, AVG(Price) AS Average_Price
    -> FROM Vegetable
    -> GROUP BY Category;
+-----------+---------------+
| Category  | Average_Price |
+-----------+---------------+
| Vegetable |     34.000000 |
| Root      |     40.000000 |
+-----------+---------------+
2 rows in set (0.03 sec)
```

HAVING

1. **Display categories whose total quantity is greater than 100.**

```
SELECT Category, SUM(Quantity) AS Total_Quantity
FROM Vegetable
GROUP BY Category
HAVING SUM(Quantity) > 100;
```

```
mysql> SELECT Category, SUM(Quantity) AS Total_Quantity
    -> FROM Vegetable
    -> GROUP BY Category
    -> HAVING SUM(Quantity) > 100;
+-----------+----------------+
| Category  | Total_Quantity |
+-----------+----------------+
| Vegetable |            540 |
+-----------+----------------+
1 row in set (0.04 sec)
```

2. **Display categories whose average price is greater than 30.**

```
SELECT Category, AVG(Price) AS Average_Price
FROM Vegetable
GROUP BY Category
HAVING AVG(Price) > 30
```

```
mysql> SELECT Category, AVG(Price) AS Average_Price
    -> FROM Vegetable
    -> GROUP BY Category
    -> HAVING AVG(Price) > 30;
+-----------+---------------+
| Category  | Average_Price |
+-----------+---------------+
| Vegetable |     34.000000 |
| Root      |     40.000000 |
+-----------+---------------+
2 rows in set (0.03 sec)
```

ORDER BY

1. Display vegetables in ascending order of price.
SELECT * FROM Vegetable

ORDER BY Price ASC;

```
mysql> SELECT * FROM Vegetable
    -> ORDER BY Price ASC;
+-------+---------+-----------+-------+----------+
| VegID | VegName | Category  | Price | Quantity |
+-------+---------+-----------+-------+----------+
|   102 | Potato  | Vegetable | 25.00 |      200 |
|   101 | Tomato  | Vegetable | 30.50 |      100 |
|   104 | Onion   | Vegetable | 35.00 |      150 |
|   103 | Carrot  | Root      | 40.00 |       80 |
|   105 | Brinjal | Vegetable | 45.50 |       90 |
+-------+---------+-----------+-------+----------+
5 rows in set (0.00 sec)
```

3. Display vegetables in descending order of price.
SELECT * FROM Vegetable

ORDER BY Price DESC;

```
mysql> SELECT * FROM Vegetable
    -> ORDER BY Price DESC;
+-------+---------+-----------+-------+----------+
| VegID | VegName | Category  | Price | Quantity |
+-------+---------+-----------+-------+----------+
|   105 | Brinjal | Vegetable | 45.50 |       90 |
|   103 | Carrot  | Root      | 40.00 |       80 |
|   104 | Onion   | Vegetable | 35.00 |      150 |
|   101 | Tomato  | Vegetable | 30.50 |      100 |
|   102 | Potato  | Vegetable | 25.00 |      200 |
+-------+---------+-----------+-------+----------+
5 rows in set (0.00 sec)
```

3. **Display vegetables in ascending order of quantity.**

SELECT * FROM Vegetable
ORDER BY Quantity ASC;

```
mysql> SELECT * FROM Vegetable
    -> ORDER BY Quantity ASC;
+-------+---------+-----------+-------+----------+
| VegID | VegName | Category  | Price | Quantity |
+-------+---------+-----------+-------+----------+
|   103 | Carrot  | Root      | 40.00 |       80 |
|   105 | Brinjal | Vegetable | 45.50 |       90 |
|   101 | Tomato  | Vegetable | 30.50 |      100 |
|   104 | Onion   | Vegetable | 35.00 |      150 |
|   102 | Potato  | Vegetable | 25.00 |      200 |
+-------+---------+-----------+-------+----------+
5 rows in set (0.03 sec)
```